

LED Asset Download and Synchronisation Application

Business Requirements Document

Version: 1.2

Date: September 2026

Application Type: Windows Desktop Application (Venue-Deployed)

Status: Approved for Development

Prepared for IT / Venue Operations — Sports Presentation Asset Distribution
Companion document to the Asset Management Web Application BRD (v2.4)

Table of Contents

- 1. Executive Summary**
- 2. Business Objective**
- 3. Scope**
 - 3.1 In Scope
 - 3.2 Out of Scope
- 4. Relationship to the Asset Management Web Application**
- 5. Technical Architecture**
- 6. User Authentication**
 - 6.1 Initial/Fixed User
 - 6.2 Login
- 7. Event Management and Dashboard**
 - 7.1 Event Information
 - 7.2 Event Dashboard
- 8. New Event Registration**
- 9. Event GUID Verification**
 - 9.1 _GUID.json
 - 9.2 GUID Validation Rules
- 10. Event Configuration JSON**
- 11. LED Asset Structure**
- 12. Device and Folder Mapping**
- 13. Device Connectivity Testing**
- 14. Application Settings**
- 15. Asset Download and Synchronisation (On-Demand)**
- 16. Cloud Storage Asset Structure**
- 17. Local Change Log**
- 18. Incremental Download Logic**
- 19. Local Asset Storage**
- 20. Push / Synchronise to LED Devices**
- 21. Error and Exception Handling**
- 22. Scheduled Download and Synchronisation**
- 23. Email Notification**
- 24. Application Dashboard (Operational View)**

- 25. Application Logging**
- 26. SQLite Database Requirements**
- 27. Installation Requirements**
- 28. New Installation vs. Application Upgrade**
- 29. First-Time Installation Workflow**
- 30. Standard Download and Sync Workflow**
- 31. Scheduled Workflow**
- 32. Functional Requirements Summary**
- 33. Non-Functional Requirements**
- 34. Business Rules**
- 35. Acceptance Criteria**
- 36. Open Points / Requirements to Confirm**
- 37. Recommended Application Navigation**
- 38. Overall Requirement**

1. Executive Summary

This document defines the requirements for the **LED Asset Download and Synchronisation Application**, a Windows desktop application that downloads sports presentation assets from Azure Storage and distributes them to LED/display devices configured at an event venue.

The application is the downstream, venue-deployed counterpart to the Asset Management Web Application: the web application manages event setup and asset upload into Azure Storage, and this desktop application downloads and synchronises those assets to the physical LED displays at the event.

The application will provide an authenticated interface through which an authorised administrator can:

- View configured sporting events and their last-updated status.
- Register and map new events, validating the event's identity via GUID.
- Download presentation assets from Azure Storage on demand.
- Synchronise downloaded assets to configured LED/display devices.
- Schedule automated download and synchronisation operations.
- Maintain local records of events, configurations, and download/synchronisation history.
- Test connectivity to configured display device folders.
- Record download, synchronisation, and exception/error information.
- Configure storage, scheduling, timestamp, and email notification settings.

The solution uses a Python backend (Flask), a Windows desktop interface, and SQLite as the local database.

2. Business Objective

The primary objective is to provide a controlled and reliable mechanism for distributing sports presentation assets from the Azure Storage environment managed by the Asset Management Web Application to the appropriate LED/display systems at an event.

The application should reduce manual file handling and provide:

- Centralised event configuration, sourced from the web application.
- Controlled, on-demand asset downloads.
- Incremental downloads based on asset change information.
- Automated distribution to mapped LED devices.
- Scheduled synchronisation.
- Clear auditability of downloaded and synchronised files.
- Device/folder connectivity verification.
- Event-level GUID validation, preventing assets from the wrong event being applied.
- Error and exception logging.

- Simple installation and application update management, without loss of operational history.
-

3. Scope

3.1 In Scope

- Windows desktop user interface.
- Local user authentication.
- Event management and dashboard.
- Event registration and mapping.
- Event GUID validation against Azure Storage.
- Azure Storage connectivity (read-only, via Storage Account access key).
- Asset download.
- Change-log processing (cloud and local).
- Local asset storage.
- LED/display folder mapping (IP address and Windows shared folder, configured locally).
- Device connectivity testing.
- Asset synchronisation/push to LED devices.
- Scheduled operations (Windows Task Scheduler).
- Email notifications.
- Application configuration/settings.
- SQLite database.
- Download/synchronisation logs.
- Exception/error logs.
- Application installer.
- Application update capability.

3.2 Out of Scope

Unless subsequently specified, the following are outside the initial scope:

- Editing or creating presentation assets (this is the responsibility of the Asset Management Web Application).
- Modifying files before transmission.
- Management of LED controller hardware itself.
- Content authoring.
- Event scoring or match-management functionality.
- Player/ranking management.

- Administration of the Azure Storage account, Azure AD, or Azure MySQL used by the web application.

4. Relationship to the Asset Management Web Application

This application is a direct downstream consumer of the Asset Management Web Application (the “web application”) described in its own BRD. It does not upload, rename, or validate presentation assets itself — that responsibility belongs entirely to the web application. This section records the specific dependencies between the two applications, as identified during solution architecture review.

Shared item	Produced by	Consumed by / purpose
Azure Storage folder structure (year container, event folder EventID - EventName , table folders, LED folders Inner/Outer/MainLED , rpi folder)	Web application (Sections 4 and 6 of its BRD)	This application mirrors the same structure locally and downloads from it (Sections 16, 19).
_ledassetchange log.csv (event-level change log)	Web application (Section 24 of its BRD)	Downloaded first on every operation to identify new/updated files (Sections 16, 18).
_GUID.json (event export/identity file)	Web application Export Event function (Section 25.6 of its BRD)	Downloaded to validate the event's identity before registration/download proceeds (Section 9).
Per-table Inner/Outer/Main LED configuration	Web application (Sections 3.1 and 5 of its BRD), carried in _GUID.json's tables field	Determines which LED folders/types this application expects to find and download (Section 11).
Storage Account access key	Provisioned at the Azure Storage account level (referenced in Section 1 of the web BRD)	Used by this application, configured under Application Settings (Section 14), to read the container directly. This application never uses Azure AD or Managed Identity.

Shared item	Produced by	Consumed by / purpose
<code>champion_winner.mp4</code> (event-specific OVR Trigger asset, example only)	Web application (Section 26 of its BRD)	No functional impact. This application's download/synchronisation logic (Sections 16-20) is filename-agnostic and already processes whatever files exist under a table/LED folder; <code>champion_winner.mp4</code> flows through unchanged, alongside <code>winner.mp4</code> , with no code change required.
<code>sponsorsequence.csv</code> (per table, per Inner/Outer LED folder, including the <code>duration</code> column)	Web application (Sections 7.1, 13, 14 of its BRD, v2.4)	No functional impact. As of web BRD v2.4, this file resides directly inside each table's Inner/Outer LED folder rather than the event folder, so it is downloaded and pushed to the mapped LED shared folder (Sections 16-20) by this application's existing, unmodified logic, in exactly the same way as any other file in that folder. This resolves the <code>sponsorsequence.csv</code> delivery-path open point raised during the earlier solution architecture review.

This application only ever reads from Azure Storage. It must never write, rename, or delete any file within the web application's Azure Storage structure; all writes performed by this application are local (SQLite, local change log, local asset copies) or to the mapped LED shared folders on the venue network.

The web application's Section 27 (Sponsor Ad Duration), Section 29 (Change Log Email Notification), Section 30 (Event Log Tab), and Section 31 (LED Resolution Configuration) additions require no code change in this application. Section 27's duration values reach the venue simply because `sponsorsequence.csv` (which now carries them) is downloaded and pushed like any other file, per the row above — no duration-specific handling exists or is required in this application.

Because the Storage Account access key grants broad read access to the storage account, it must be treated as a sensitive configuration value and protected under Section 33 (Security) even though the fixed local login itself is not similarly protected (see Section 6.1).

5. Technical Architecture

Component	Technology
Application Type	Windows Desktop Application
Frontend	Windows desktop UI
Backend	Python
Backend Framework	Flask
Local Database	SQLite
Cloud Storage	Azure Blob Storage (the Storage Account managed by the web application)
Local File System	Windows folders
Device Distribution	Windows network/shared folders
Scheduling	Windows Task Scheduler, invoking the application/a batch process
Notifications	Email/SMTP configuration
Configuration	JSON event configuration, downloaded per event from Azure Storage

The architecture should separate the user interface, business logic, storage services, synchronisation services, and database operations, to allow future expansion.

6. User Authentication

6.1 Initial/Fixed User

Parameter	Requirement
Username	admin
Password	Admin@123
Role	Administrator

The application maintains a single fixed administrative login; there is no per-user Azure AD integration, and no separate roles are required within this application.

It has been explicitly decided that this credential may be stored in plain text within the application's local configuration. This is an accepted risk, based on the application running only for the duration of an event on a venue-local machine, rather than a general security recommendation. The credential must remain changeable by the administrator.

6.2 Login

The application shall display a login screen when launched. The user must successfully authenticate before accessing the application.

Login flow:

- 1 Launch application.
 - 2 Display login screen.
 - 3 Enter username.
 - 4 Enter password.
 - 5 Validate credentials.
 - 6 On successful authentication, display the application dashboard.
 - 7 On failure, display an appropriate authentication error.
-

7. Event Management and Dashboard

The application will maintain a local register of events in SQLite.

7.1 Event Information

Each registered event should contain, at minimum:

- Event ID
- Event Name
- Event GUID (as validated at registration, Section 9)
- Event configuration reference (the source `_GUID.json` location in Azure Storage)
- Event configuration JSON (downloaded content, Section 10)
- Last asset update timestamp
- Last download timestamp
- Last synchronisation timestamp
- Event status

7.2 Event Dashboard

After login, the application shall display the configured events.

Field	Description
Event ID	Unique event identifier
Event Name	Name of event
Last Updated	Date/time of latest relevant asset update
Status	Current event/configuration status

Date/time format: the application shall use **DD/MM/YY HH:MM** on the 24-hour clock, for example **08/09/26 19:45**.

8. New Event Registration

The application must support registration of a new event. A new installation may initially contain an empty SQLite database (Section 28.1).

Registration process:

- 1 Administrator selects Add New Event.
- 2 Administrator enters/provides the event information (at minimum, the Event ID).
- 3 Application downloads the event's **_GUID.json** configuration from Azure Storage.
- 4 Application identifies the event GUID within that configuration.
- 5 Application validates the GUID (Section 9).
- 6 Event is registered in SQLite.
- 7 Event configuration is associated with the event record.
- 8 Event becomes available for subsequent download/synchronisation operations.

An event must be successfully registered, with its GUID validated, before it can be entered for use — only then can download operations proceed for that event.

9. Event GUID Verification

GUID verification is a mandatory control for event registration and every subsequent download. It exists to prevent assets belonging to another event from being downloaded or synchronised accidentally.

9.1 **_GUID.json**

The event's folder in Azure Storage will contain a file named:

_GUID.json

This file is produced and kept up to date by the web application's Export Event function (Section 25.6 of the web application's BRD) and contains, among other fields, the event's current **exportGuid**.

9.2 GUID Validation Rules

The application shall:

- 1 Read the GUID recorded against the event locally (from the event's initial registration, Section 8).
- 2 Retrieve the current **_GUID.json** from Azure Storage.
- 3 Compare the two GUID values.
- 4 Permit the operation to proceed only when the GUID values match.
- 5 Reject the operation when the GUID values do not match.
- 6 Record the failure in the exception log (Section 21.1).

Because the web application generates a new ExportGUID every time Export Event is used (Section 25.5 of the web application's BRD), re-running Export Event for an event after this application has already registered it will cause subsequent GUID validation to fail until the event is re-registered/re-mapped in this application. This sequencing dependency should be clearly communicated to venue operators.

10. Event Configuration JSON

Each event will have a JSON configuration file (**_GUID.json**, Section 9.1), downloaded from Azure Storage and associated with the corresponding event record in SQLite.

Per the web application's export schema (Section 25.4 of its BRD), the configuration provides:

- Event ID (**eventId**)
- Event name (**eventName**)
- Event storage location (**eventStorageUrl**)
- Per-table LED type configuration (**tables**: table number plus Inner/Outer/Main LED Booleans)
- Export metadata (exported by, role, timestamp)
- Event GUID (**exportGuid**)

Device mappings (IP addresses, Windows shared folders) are **not** part of this configuration — they are venue-specific network details that only this application knows, and are configured locally as described in Section 12.

11. LED Asset Structure

An event may contain one or more tables, for example:

- **Table 1**

- Table 2
- Table 3

Each table may contain one or more LED/display types. The supported LED/display types are:

- Inner
- Outer
- MainLED

Therefore, a possible structure could be:

Event

```
|-- Table 1
|   |-- Inner
|   |-- Outer
|   `-- MainLED
|-- Table 2
|   |-- Inner
|   `-- Outer
`-- Table 3
    `-- MainLED
```

The actual structure for a given event shall be determined entirely by the downloaded `_GUID.json` event configuration (Section 10), which reflects the LED configuration set on the web application (Sections 3.1 and 5 of its BRD) — this application does not independently decide which LED types exist for a table.

The specific asset filenames found under a table/LED folder (for example `winner.mp4` or the event-specific `champion_winner.mp4`, per Section 26 of the web BRD) are not inspected or hard-coded by this application — any file present is downloaded and synchronised the same way, per Section 4.

As of web BRD v2.4, this includes `sponsorsequence.csv`, which now exists once per table per Inner/Outer folder (rather than once per event) and is downloaded and synchronised identically to any other file found in those folders — see Section 4.

12. Device and Folder Mapping

Each LED type enabled for a table shall be mapped, locally within this application, to a Windows network/shared folder and an IP address.

For example:

Table	LED Type	IP Address	Shared Folder	Status
Table 1	Inner	Configured IP	Configured share	Tested
Table 1	Outer	Configured IP	Configured share	Tested

Table	LED Type	IP Address	Shared Folder	Status
Table 1	Main LED	Configured IP	Configured share	Tested
Table 2	Inner	Configured IP	Configured share	Tested

Only the LED types enabled for a table in the event configuration (Section 10) shall be presented for mapping — for example, if Table 1 has only Inner and Outer enabled, only Inner and Outer mapping fields shall be shown for Table 1.

The mapping shall be retained locally (SQLite, Section 26) and used for every subsequent connectivity test (Section 13) and push/synchronise operation (Section 20).

13. Device Connectivity Testing

The application shall provide a Test Connection function for every configured LED/display destination.

The test shall verify access to the corresponding network/shared folder — not merely to the device or server in general.

Test process:

- 1 Read configured IP/device information.
- 2 Identify the mapped folder.
- 3 Attempt to access the folder.
- 4 Verify that the application has the required access.
- 5 Return success/failure status.
- 6 Record the result in the log.

Example: Table 1 → Inner → Connection Successful or Table 1 → Outer → Connection Failed.

14. Application Settings

The application shall provide a dedicated Settings section, covering the following configuration parameters.

Cloud Storage Settings

- Storage account
- Storage container
- **Storage Account access key** (Section 4)
- Event storage path
- Asset storage path

Download Settings

- **Last Updated Timestamp Cut-off** (Section 18)
- Download location
- Change-log location
- Download retry configuration
- Synchronisation behaviour

Scheduling Settings

- Enable/disable scheduled operation
- Scheduled day(s)
- Scheduled time (24-hour format)
- Windows Task Scheduler / batch execution option

Email Settings

- IT notification email address
- SMTP server
- SMTP port
- SMTP username
- SMTP authentication configuration
- Notification enable/disable

Application Settings

- Application logging location
- Exception logging location
- Database location
- Update/installer configuration

15. Asset Download and Synchronisation (On-Demand)

The administrator shall be able to manually initiate **Download & Sync** for a registered event.

The process shall:

- 1 Validate the event configuration (Section 10).
- 2 Validate the event GUID (Section 9).
- 3 Connect to Azure Storage using the configured Storage Account access key (Section 14).
- 4 Download the applicable `_ledassetchange.log.csv` change log (Section 16).
- 5 Identify new/updated assets (Section 18).
- 6 Download the required assets.

- 7 Store the assets locally, mirroring the cloud structure (Section 19).
 - 8 Validate the download.
 - 9 Push the assets to the mapped LED/display folders (Section 20).
 - 10 Record the operation in the local change log (Section 17).
 - 11 Update the event's last-updated information.
 - 12 Record exceptions where applicable (Section 21).
-

16. Cloud Storage Asset Structure

The application shall retrieve the event's asset change information from `_ledassetchange.log.csv`, located directly beneath the event's folder in Azure Storage, as maintained by the web application (Section 24 of its BRD).

The application shall retrieve this change log and process its entries to determine which files need to be downloaded.

As of web BRD v2.4, the same filename (most notably `sponsorsequence.csv`) can appear once per table per Inner/Outer folder. The web application's change log entries carry enough path context (for example, `Table 1/Inner/sponsorsequence.csv`) to identify the specific file and folder affected, per Section 24 of its BRD; this application must use that full path when determining which local file to update, consistent with the Table and LED Type columns already recorded in its own local change log (Section 17).

17. Local Change Log

The application shall maintain its own local change log, separate from `_ledassetchange.log.csv`, recording what this application has itself downloaded.

The local change log shall be maintained as a CSV file, recommended filename `_localchange.log.csv`, stored in the application's root/application data directory.

Each record should contain:

Field	Description
Serial Number	Unique log record
Event ID	Associated event
Table	Table number
LED Type	Inner / Outer / Main LED

Field	Description
File Name	Downloaded asset
Source Location	Azure Storage path
Local Location	Local file path
Timestamp	Processing timestamp
Source Updated Timestamp	Asset's timestamp in <code>_ledassetchange.log.csv</code>
Download Status	Success/Failure
Sync Status	Success/Failure

Example: `001, EVENT001, Table 1, Inner, asset_001.png, 08/09/26 19:32`

18. Incremental Download Logic

The application shall not unnecessarily download every asset during every synchronisation.

The application shall compare the latest `_ledassetchange.log.csv` against the local change log (Section 17). Only files that are new, or updated since the last successful processing, shall be downloaded.

The Last Updated Timestamp Cut-off is configurable through Application Settings (Section 14).

19. Local Asset Storage

Downloaded assets shall be stored locally using the same logical folder structure as Azure Storage, for example:

Application Data

```

  -- Event
    -- Table 1
      -- Inner
      -- Outer
      -- Main LED
    -- Table 2
      -- Inner
      -- Main LED

```

The local structure shall correspond to the event configuration (Section 10) and the Azure Storage structure (Section 16).

20. Push / Synchronise to LED Devices

Once assets have been successfully downloaded, the application shall synchronise them to the mapped Windows network/shared folders (Section 12).

Synchronisation process, for each successfully downloaded file:

- 1 Determine table.
- 2 Determine LED type.
- 3 Retrieve configured destination.
- 4 Verify destination accessibility.
- 5 Copy/push the file.
- 6 Verify successful transfer.
- 7 Update synchronisation status.
- 8 Record the result.

Example:

Table 1

Inner → \\Device01\Inner

Outer → \\Device01\Outer

Main LED → \\Device02\MainLED

Different LED types may map to the same physical device while using different shared folders.

21. Error and Exception Handling

The application shall clearly distinguish between:

- Download errors
- Azure Storage connectivity errors
- GUID validation errors
- Configuration errors
- File access errors
- Network/device connectivity errors
- Synchronisation errors
- Permission errors
- Missing folder errors
- Invalid configuration errors

21.1 Exception Log

A separate exception/error log shall be maintained. Each record should include:

- Date/time
- Event ID
- Table
- LED type
- File name
- Operation
- Error category
- Error description
- Source
- Destination
- Resolution/status

The application should allow the administrator to review errors after an operation.

22. Scheduled Download and Synchronisation

The same Download & Sync process (Section 15) shall support scheduled execution.

The administrator shall be able to configure: Scheduled day, Scheduled time, Enable/disable schedule (Section 14).

The scheduled operation shall run using **Windows Task Scheduler**, invoking the application or a batch process at the configured time; this application does not run as a background Windows service.

Scheduled process, at the configured time:

- 1 Windows Task Scheduler triggers the configured application/batch process.
 - 2 Application starts the download/synchronisation process.
 - 3 Event configuration is validated.
 - 4 GUID is validated.
 - 5 Change log is processed.
 - 6 New/updated assets are downloaded.
 - 7 Assets are synchronised to mapped destinations.
 - 8 Logs are updated.
 - 9 Errors are recorded.
 - 10 Email notification is generated where configured (Section 23).
-

23. Email Notification

The application shall provide email notification to the configured IT contact (Section 14). Email notifications should be generated following scheduled operations.

The notification should contain:

- Event name
- Event ID
- Operation date/time
- Number of files identified
- Number downloaded
- Number synchronised
- Number failed
- Error summary
- Overall operation status

Example statuses: **Successful**, **Successful with Exceptions**, **Failed**.

24. Application Dashboard (Operational View)

The main dashboard should provide an operational overview, in addition to the Event List (Section 7.2).

Recommended components:

- Event List (Event ID, Event Name, Last Updated, Status, Actions)
 - Actions: Add Event, Download & Sync, Test Connections, View Logs, Settings
 - Operation Status: current operation, progress, files identified, files downloaded, files synchronised, errors
-

25. Application Logging

The application should maintain operational logs independently from the change log (Section 17) and the exception log (Section 21.1).

The operational log should capture: application startup, login, event creation, configuration changes, download operations, synchronisation operations, scheduled executions, device tests, errors, and application updates.

All operational log timestamps should use **DD/MM/YY HH:MM:SS**, since detailed logging requires seconds (compare the coarser **DD/MM/YY HH:MM** display format used on the Event Dashboard, Section 7.2).

26. SQLite Database Requirements

SQLite shall be the local application database. A suggested logical database structure is:

events

event_id, event_name, event_guid, configuration_file, configuration_version, last_updated, last_download, last_sync, status

led_mappings

mapping_id, event_id, table_number, led_type, ip_address, shared_folder, enabled, last_connection_test, connection_status

download_history

download_id, event_id, file_name, table_number, led_type, source_path, local_path, source_timestamp, download_timestamp, status

sync_history

sync_id, event_id, file_name, destination, sync_timestamp, status, error_message

application_settings

setting_name, setting_value

operation_log

log_id, event_id, operation, timestamp, status, message

exception_log

exception_id, event_id, timestamp, operation, category, message, resolution_status

27. Installation Requirements

The application shall be distributed through a Windows installer. The installer shall:

- Install the application on a Windows machine.
 - Create required application directories.
 - Create the SQLite database.
 - Install required application components.
 - Create desktop/start-menu shortcuts where required.
 - Configure required Windows Task Scheduler components.
 - Support application upgrades.
-

28. New Installation vs. Application Upgrade

When the installer is executed, it shall determine whether the installation is new or an upgrade.

28.1 New Installation

- Application files are installed.
- SQLite database is created/initialised, with no existing events.
- Application/user configuration is initialised.
- Application is ready for first-time event registration.

28.2 Existing Installation / Upgrade

- Existing SQLite database shall be retained.
- Existing event mappings shall be preserved.
- Existing operational history should be retained.
- Only application code/components should be updated, unless a database migration is explicitly required.

The installer must not clear the SQLite database during an application upgrade.

29. First-Time Installation Workflow

Install Application → Initialise SQLite → Login → Configure Application Settings → Configure Cloud Storage → Register Event → Download Event Configuration (`_GUID.json`) → Validate GUID → Register Event in SQLite → Load LED/Table Configuration → Configure Device/Folder Mapping → Test Connections → Ready for Download & Sync

30. Standard Download and Sync Workflow

User selects Event → Validate Configuration → Validate GUID → Connect to Azure Storage → Retrieve `_ledassetchange.log.csv` → Compare with Local Change Log → Identify New/Updated Files → Download Files → Validate Downloads → Update Local Change Log → Push to LED Shared Folders → Verify Synchronisation → Update SQLite → Update Logs → Display Result

31. Scheduled Workflow

Windows Task Scheduler → Trigger Application/Batch → Load Configuration → Validate Event/GUID → Retrieve Change Log → Identify Changes → Download Assets → Synchronise Assets → Update Database/Logs → Generate Result → Send IT Email

32. Functional Requirements Summary

ID	Requirement	Priority
FR-001	Windows desktop application	Must
FR-002	Python backend	Must
FR-003	Flask backend framework	Must
FR-004	SQLite database	Must
FR-005	Local user authentication (fixed admin account)	Must
FR-006	Event management and dashboard	Must
FR-007	Event GUID validation against <code>_GUID.json</code>	Must
FR-008	JSON event configuration, downloaded from Azure Storage	Must
FR-009	Multiple tables supported	Must
FR-010	Inner/Outer/Main LED types supported	Must
FR-011	Device/folder mapping (local, manual)	Must
FR-012	Connection testing per destination folder	Must
FR-013	On-demand download and sync	Must
FR-014	Asset synchronisation/push to LED devices	Must
FR-015	Incremental download logic	Must
FR-016	Local change log	Must
FR-017	Exception log	Must
FR-018	Scheduled operation via Windows Task Scheduler	Must
FR-019	Email notification	Should
FR-020	Application settings	Must
FR-021	Windows installer	Must
FR-022	New installation handling	Must
FR-023	Application upgrade handling	Must
FR-024	Preserve database during upgrades	Must

ID	Requirement	Priority
FR-025	Operational logging	Must

33. Non-Functional Requirements

33.1 Reliability

The application should ensure that an incomplete or failed download does not result in an incomplete synchronisation to an LED destination.

33.2 Data Integrity

Files should be validated before being marked as successfully downloaded or synchronised.

33.3 Security

The Storage Account access key must not be stored as plain text where avoidable. The fixed local login credential is a documented exception to this principle (Section 6.1).

33.4 Usability

The application should provide clear status messages, distinguishing between In progress, Successful, Failed, Warning, and Completed with exceptions.

33.5 Recoverability

If an operation fails, the administrator should be able to rerun the operation without corrupting the existing local asset structure.

33.6 Auditability

All significant download, synchronisation, and configuration activities should be logged.

34. Business Rules

- 1 An event must be registered before assets can be downloaded.
- 2 An event GUID must be successfully validated before processing.
- 3 Assets must first be downloaded successfully before being synchronised to LED destinations.
- 4 Only new or changed files should be processed during incremental synchronisation.
- 5 LED mappings must correspond to the downloaded event configuration.
- 6 Each configured LED destination must be connection-tested.

- 7 Failed destinations must be recorded as exceptions.
 - 8 A failed file transfer must not be recorded as successfully synchronised.
 - 9 Application upgrades must preserve the existing SQLite database.
 - 10 A new installation may initialise an empty SQLite database.
 - 11 Scheduled operations must use the configured schedule.
 - 12 Email notifications must use the configured IT email destination.
 - 13 This application must never write, rename, or delete files within the web application's Azure Storage structure (Section 4).
-

35. Acceptance Criteria

AC-01 — Login

Administrator can launch the application and successfully authenticate.

AC-02 — Event Display

Registered events are displayed with Event ID, Event Name, and Last Updated timestamp.

AC-03 — New Event

Administrator can register a new event.

AC-04 — GUID Validation

Application rejects an event when the Azure Storage GUID does not match the locally recorded GUID.

AC-05 — Configuration

Application correctly reads `_GUID.json` and identifies tables and LED types.

AC-06 — Device Mapping

Administrator can map an LED type to an IP/shared folder.

AC-07 — Connection Test

Application can successfully test an accessible destination and report an inaccessible destination as failed.

AC-08 — Download

Application downloads new/updated assets from Azure Storage.

AC-09 — Incremental Download

Previously processed unchanged files are not unnecessarily downloaded.

AC-10 — Synchronisation

Successfully downloaded files are copied to their configured LED destinations.

AC-11 — Error Handling

Download and synchronisation errors are recorded in the exception log.

AC-12 — Scheduled Operation

Configured scheduled operation executes automatically via Windows Task Scheduler.

AC-13 — Notification

Configured IT recipient receives an operation status email.

AC-14 — New Installation

New installation creates a clean SQLite database.

AC-15 — Upgrade

Application upgrade preserves existing event configuration and database records.

36. Open Points / Requirements to Confirm

The transcript and prior draft establish the core requirements. The items below were resolved during solution architecture review by cross-referencing the Asset Management Web Application BRD; the remaining items should still be confirmed before development begins.

Resolved during review:

- Cloud storage technology: Azure Blob Storage (the same Storage Account used by the web application).
- Event folder/file naming convention: matches the web application's structure exactly (Sections 4, 6, 11, 16, 19 of this document).
- Name and schema of the event configuration/GUID file: `_GUID.json`, schema defined in Section 25.4 of the web application's BRD (Section 9–10 of this document).
- Name of the cloud asset change log: `_ledassetchange.log.csv` (Section 16).
- Change log format: CSV, both in the cloud and locally (Sections 16, 17).
- Whether the scheduled process needs to run as a Windows service: no — it runs via Windows Task Scheduler invoking the application/a batch process (Section 22).

Still open — to confirm before development begins:

- Exact definition of the Last Updated Timestamp Cut-off, and whether timestamps are stored/compared in local event time or UTC.
- Maximum number of supported tables per event.
- Whether IP addresses need to be ping-tested in addition to shared-folder accessibility.
- Authentication method for the Windows shared folders (e.g. domain credentials, local venue credentials, or unauthenticated LAN access).

- SMTP/email service to be used for IT notifications.
- Whether the scheduled process should run when no user is logged into Windows (this affects whether Windows Task Scheduler is configured to “run whether user is logged on or not”).
- Retry behaviour for failed downloads and failed LED synchronisation.
- Whether files should be checksum-validated after download and after push to the LED destination.
- Whether old local assets should ever be deleted, and if so, under what retention rule.
- Application update distribution location.
- Whether database schema migrations will be required during future upgrades.
- Whether multiple administrator users will eventually be supported (see Section 6.1's accepted-risk single fixed credential).
- Local change log filename: this document recommends `_localchange log.csv`; to be confirmed.

37. Recommended Application Navigation

Login

```

  |-- Dashboard
    |-- Events
      |-- Event Details
      |-- LED Configuration
      |-- Device Mapping
    |-- Download & Sync
    |-- Schedule
    |-- Logs
      |-- Download Log
      |-- Synchronisation Log
      |-- Exception Log
  |-- Settings
    |-- Cloud Storage
    |-- Timestamp
    |-- Schedule
    |-- Email
    |-- Application

```

38. Overall Requirement

The final application should provide a simple operational interface for a venue IT/event operator, while the underlying Python/Flask services handle Azure Storage connectivity, configuration processing, incremental asset downloading, local storage, LED-device synchronisation, scheduling, logging, and notifications.

The most important architectural principle is that the event's downloaded `_GUID.json` configuration — sourced from the Asset Management Web Application — remains the source of truth for the event's table/LED structure and identity, while SQLite maintains this application's own operational state, device mappings, and history. Device/folder mappings themselves are local-only and are never sourced from the web application.

39. Kickoff Confirmations and BRD Amendments (Addendum A)

Added: 20 September 2026 — amends LED Asset Download and Synchronisation Application BRD v1.2

This addendum records decisions the project owner confirmed on 20 September 2026, ahead of Claude Code implementation kickoff for this application. It supersedes the corresponding entries in Section 36 where noted below, and adds one new business rule referenced from Section 23 and Section 34. This BRD should be read together with this addendum; the effective document version is v1.2 + Addendum A.

39.1 Storage Account and Test Event — Resolved

- **Confirmed:** this application connects to the same Azure Storage Account already used by the web application (Section 4) — a separate Storage Account is not required.
- **Confirmed:** a real test event already exists in that Storage Account for development — Event ID 1000, “Star contender Doha” — created via a genuine Export Event run from the web application (exported 18 September 2026), with its `_GUID.json`, `_ledassetchangelog.csv`, and populated Table 1 (Inner/Outer/MainLED) and Table 2 (Inner) folders.
- **Note for Claude Code:** a local mirror of this event's files is also available at `references/0forimplementation/desktop/samplefiles-webassetmgmt/` for offline reference. That local mirror's own folder names (e.g. `Table1/inner`) are a local convenience naming only — the authoritative cloud folder naming, confirmed by the real change log, is `Table 1/Inner`, `Table 1/Outer`, `Table 1/MainLED`, exactly matching Section 11's documented convention. Build against the cloud naming, not the local mirror's casing.
- **Not reproduced here:** the actual Storage Account access key is a live credential and is intentionally not written into this document, or into any file in this `ledassetmanagement` repository. It is supplied directly by the project owner to Claude Code at Phase 4 kickoff and stored only inside the `desktopsportsres` application repository's own local configuration — see the Solution Implementation Plan, Section 1.

39.2 Venue Network for Development — Resolved (development scope only)

- **Confirmed:** local/simulated Windows shared folders will be used for Phase 5 (Device and Folder Mapping) and Phase 8 (Push/Synchronise) development, since real venue shared folders are not yet available. The project owner will supply the specific local folder paths before Phase 5 begins.
- **Still open:** the authentication method for real venue shared folders (domain credentials / local venue credentials / unauthenticated LAN access) is not resolved by this item and remains open per Section 36 — it must still be confirmed before venue deployment, and before Step 5.3 of the Implementation Sequence.

39.3 Email / SMTP Mechanism — Resolved, plus new business rule

- **Confirmed:** this application's operational email notification (Section 23) will use Azure Communication Services — the same service used by the web application's own Change Log Email Notification feature. Connection details will be supplied when Phase 11 begins.

NEW BUSINESS RULE (amends Section 23 — Email Notification, and Section 34 — Business Rules):

If the venue machine has no internet/network connectivity at the time a notification would be sent, the application must skip the email notification without treating the omission as an operation failure. The Download & Sync or scheduled operation itself must still complete and be logged normally (Sections 17, 21, 25); only the notification step is silently skipped. This skip must itself be recorded in the operational log (Section 25) as “notification skipped — no connectivity.”

39.4 Installer / Update Distribution Location — Resolved

- **Confirmed:** the installer package is produced to and stored in the desktopinstaller container within the same Storage Account (<https://sasportspresentation.blob.core.windows.net/desktopinstaller>). This resolves the “Application update distribution location” item in Section 36.
- **Confirmed:** code-signing is not required for the venue-deployed Windows executable.
- **Still open:** whether future upgrades will require database schema migrations is not resolved by this item and remains open per Section 36.

39.5 Source Control — Resolved (process, not a functional requirement)

- **Confirmed:** this application's code is committed to <https://github.com/vatsansg/desktopsportspres>, with branch strategy and commit conventions consistent with the web application's process. Recorded here for traceability only — the operative instruction for Claude Code is in the Solution Implementation Plan, Section 2.

39.6 Updated Status of Section 36 — Open Points

The table below reflects the current status of every item listed in Section 36, incorporating this addendum. It does not replace Section 36's text — read the two together.

Item	Status
Storage Account and access key confirmation	Resolved — see 39.1
Representative test event for development	Resolved — see 39.1
Venue network for development (local vs. real)	Resolved for development — see 39.2
Windows shared-folder authentication method	Still open (Section 36)
SMTP/email service to use	Resolved — see 39.3
Behaviour when no internet connectivity at notification time	Resolved (new rule) — see 39.3
Application update distribution location	Resolved — see 39.4
Code-signing requirement for installer	Resolved — see 39.4
Source repository and process	Resolved — see 39.5
Last Updated Timestamp Cut-off definition / timezone	Still open (Section 36)
Maximum tables per event	Still open (Section 36)
Ping-testing vs. shared-folder-access testing	Still open (Section 36)
Whether scheduled runs must work with no user logged in	Still open (Section 36)
Retry behaviour for failed downloads/synchronisation	Still open (Section 36)
Checksum validation after download/push	Still open (Section 36)
Local asset retention/deletion policy	Still open (Section 36)
Database schema migration approach	Still open (Section 36)

Item	Status
Multi-administrator support	Still open (Section 36)
Local change log filename confirmation	Still open (Section 36)